



RAJALAKSHMI
ENGINEERING COLLEGE

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23432 – Software Construction

(REGULATION 2023)

RAJALAKSHMI ENGINEERING COLLEGE
Thandalam, Chennai-602015

Name: Adithya.B

Register No: 240701018

Year / Branch / Section : II / CSE / A

Semester: 4

Academic Year: 2026

INDEX

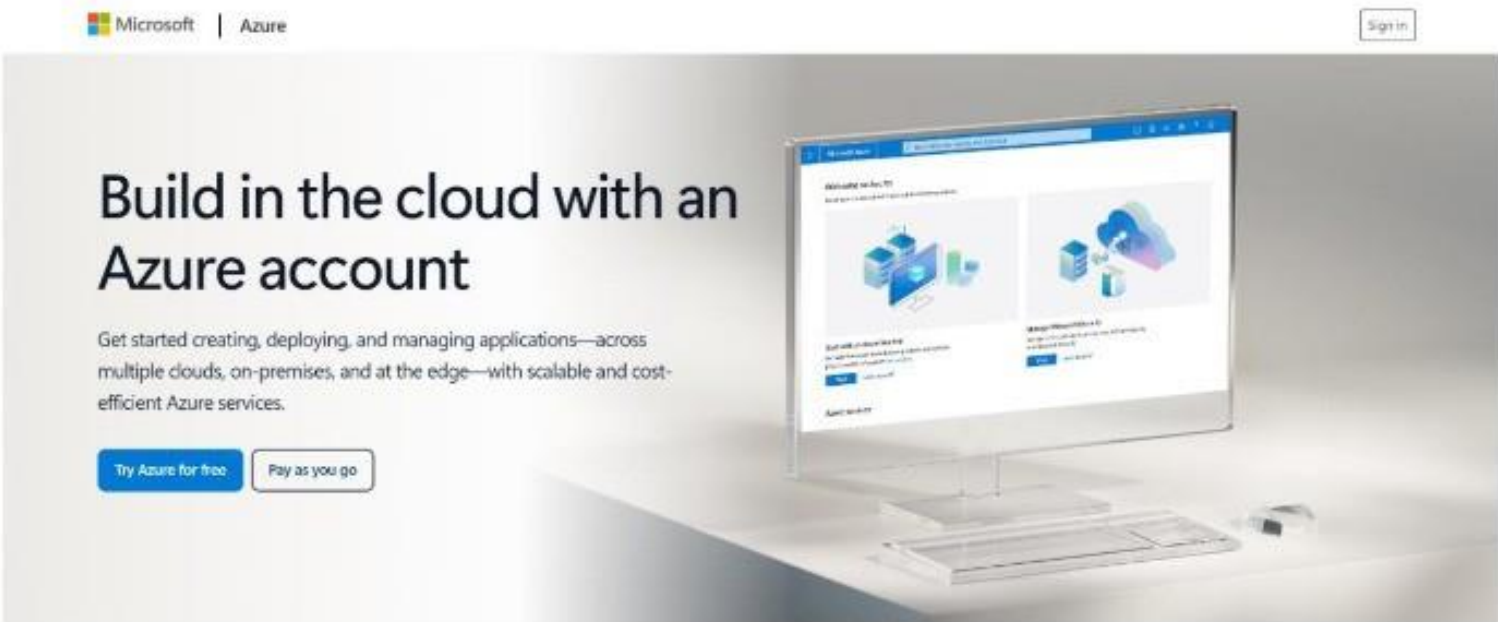
S.No.	Date	Title
1.		Azure Devops Environment Setup.
2.		Azure Devops Project Setup and User Story Management.
3.		Setting Up Epics, Features, And User Stories for Project Planning.
4.		Sprint Planning.
5.		Poker Estimation.
6.		Designing Class and Sequence Diagrams for Project Architecture.
7.		Designing Architectural and ER Diagrams for Project Structure.
8.		Testing – Test Plans and Test Cases.
9.		Load Testing and Pipelines.
10.		GitHub: Project Structure & Naming Conventions.

EXP NO:	AZURE DEVOPS ENVIRONMENT SETUP Create Epic Features User
---------	--

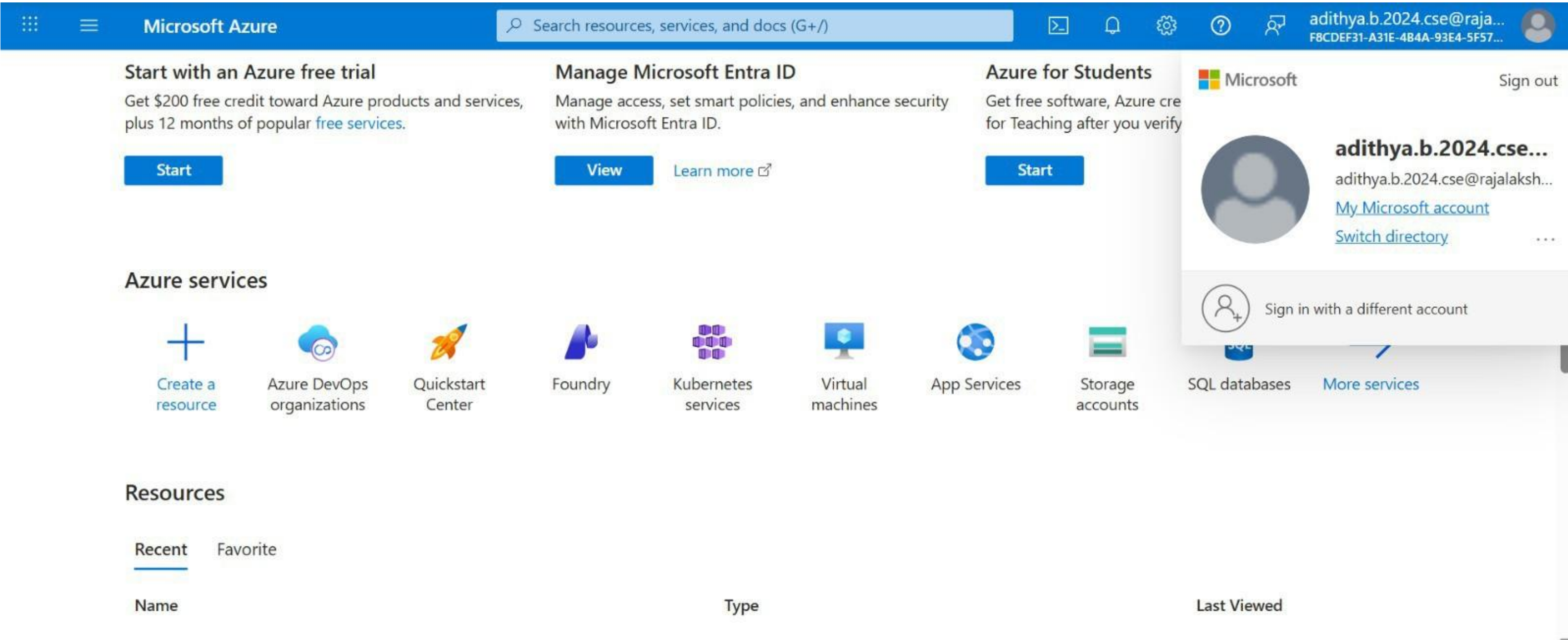
Aim:
To set up and access the Azure DevOps environment by creating an organisation through the Azure portal.

INSTALLATION

- 1.Open your web browser and go to the Azure website: <https://azure.microsoft.com/en-us/get-started/azure-portal>. Sign in using your Microsoft account credentials.
- If you don't have a Microsoft account, you can create one here: <https://signup.live.com/?lic=1>



2.Azure home page



3. Open DevOps environment in the Azure platform by typing **Azure DevOpsOrganizations** in

the search bar.

4.Click on the **My Azure DevOps Organization** link and create an org anization and you should be taken to the Azure DevO ps Organization Hom e page.

5. Click on the Azure DevOps service link from the search results

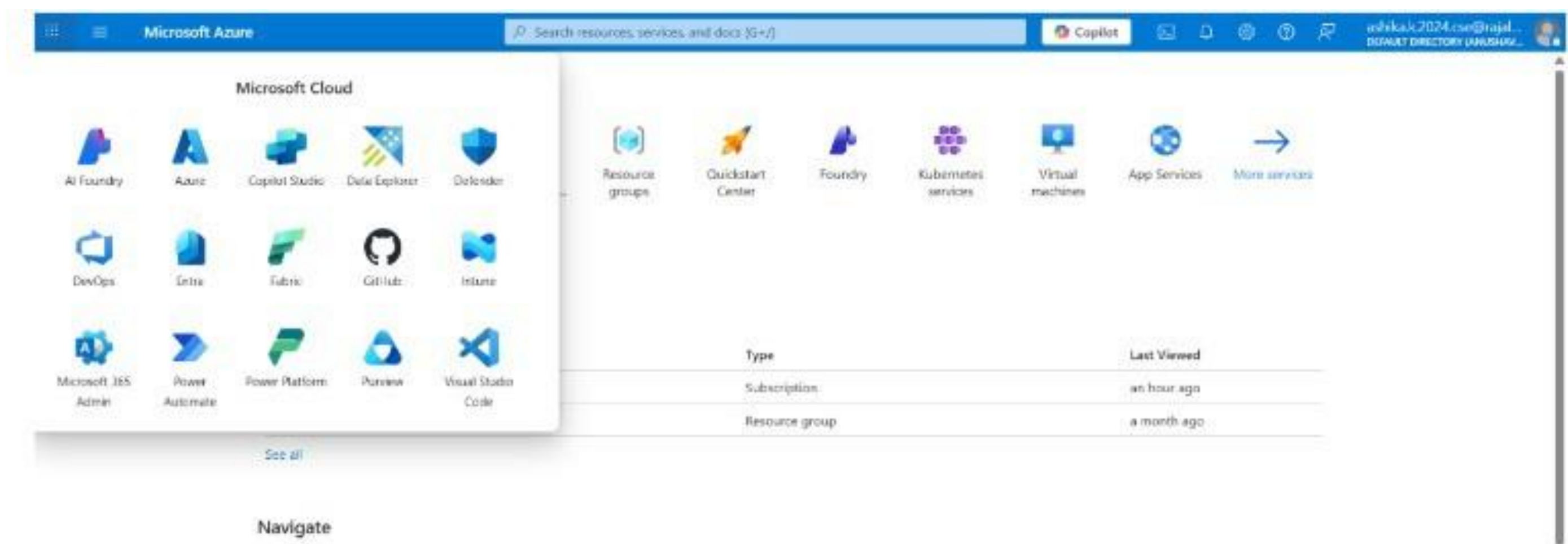
6Create a new organization by following the prompts.

7.After creation, you will be redirected to the **Azure DevOps Organization Home Page**.

8. After creating the organization, choose the **region** where your data will be hosted.

9. Accept the terms and conditions to proceed with organization creation.

10. Once inside Azure DevOps, click on “**New Project**” to create your first project



Microsoft

Adithya BSign out

AB

Adithya B

adithya.b.2024.cse@rajalakshmi.edu.in

India

adithya.b.2024.cse@rajalakshmi.edu.in

Edit profile

Visual Studio Dev Essentials

Azure DevOps Organizations

Create new organization

dev.azure.com/sivas2024cse (Member)

Projects

DIGITAL CERTIFICATE MANAGEMENT SYSTEM WITH BATCH GENERATION -1

Actions

Open in Visual Studio

Manage security

Browse extensions

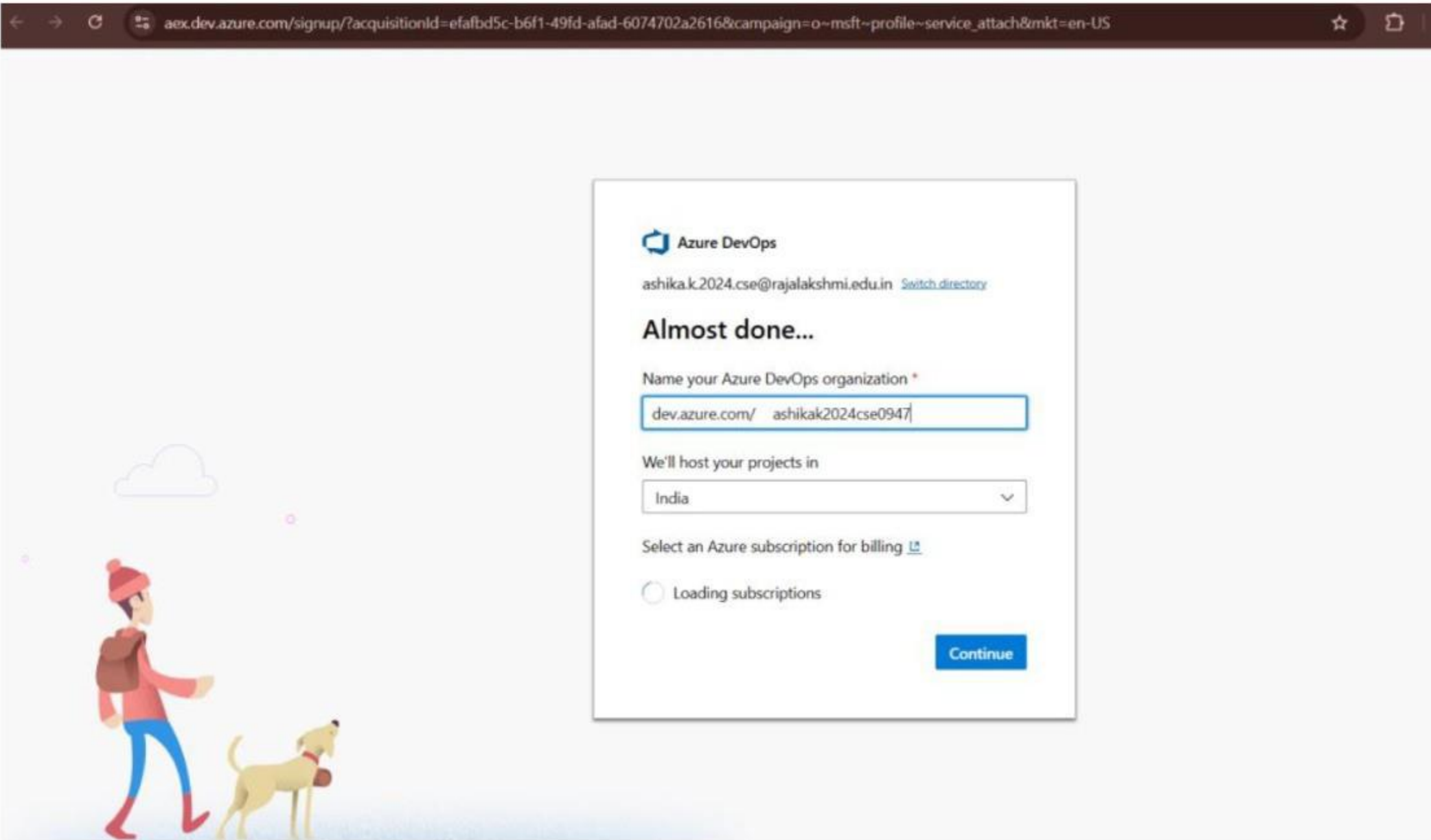
Leave

Result:
Successfully acc essed the Azure Dev Ops environment and created a new organization through the Azure portal.

EXP NO:	AZURE DEVOPS PROJECT SETUP AND USER STORY Create Epic Features User Stories
---------	---

Aim:
To set up an Azure DevOps project for efficient collaboration and agile work management.

1.Project dashboard



Create new project

Project name *

Digital Certificate Management System

Description

Visibility

Private

Only people you give access to will be able to view this. Want to create a public project? [Try GitHub](#)

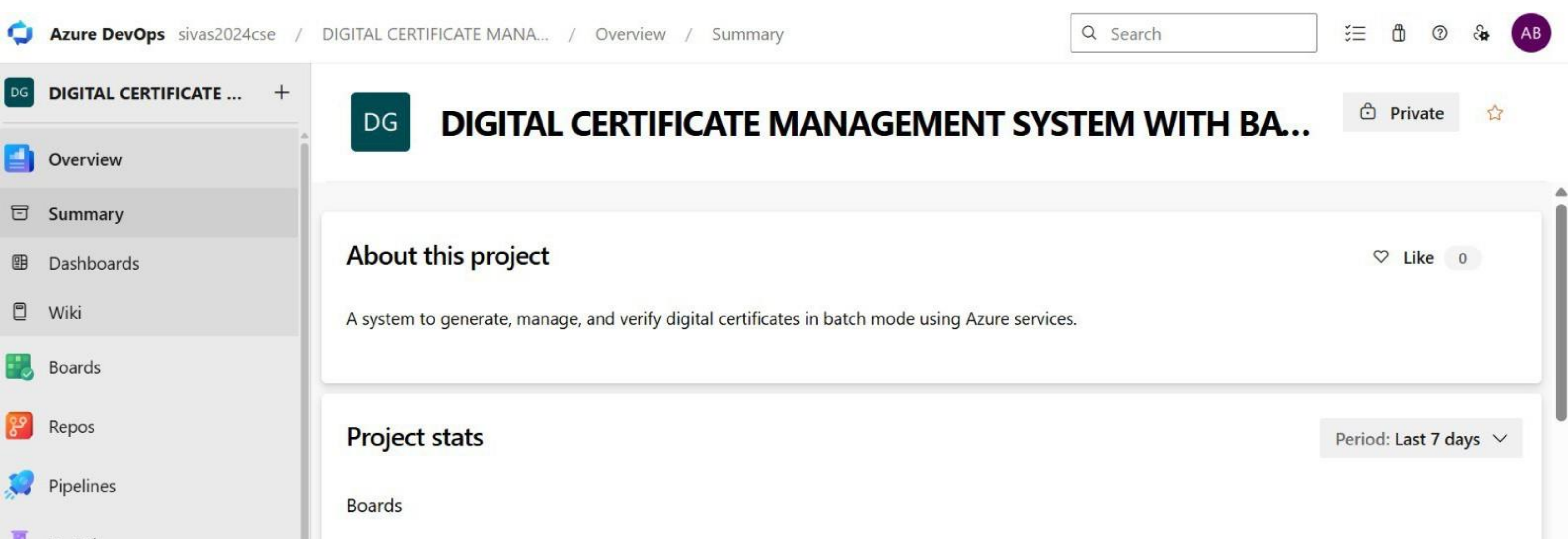
Advanced

Version control ⓘ

Git

Work item process ⓘ

Basic



1. From the left-hand navigation menu, click on **Boards**.
 - This opens the main Boards page where you can manage work items, backlogs, and sprints.
2. From the left-hand navigation menu, click on **Boards**.
 - This opens the main Boards page where you can manage work items, backlogs, and sprints.
3. Navigate to the **Work Items** section.
4. Click on “**Add a work item**” at the top of the page.
 - Alternatively, use the “**+**” **button** or “**New Work Item**” option depending on the view.
5. From the dropdown menu, select **User Story**.

6. Enter the required details:
 - Title
 - Description
 - Acceptance Criteria
 - Priority
 - Assigned To
7. Click **Save** to create the user story.
8. Organize user stories in the **Backlog** to prioritize work.
9. Use the **Kanban Board** to track progress:
10. Create **Sprints** to plan iterative development cycles.
11. Assign user stories to specific sprints for better tracking.
12. Break down user stories into smaller **Tasks** for easier execution.
13. Link **Bugs** to user stories for issue tracking.
14. Use **Tags** to categorize and filter work items.
15. Enable **Notifications** to keep team members updated.
16. Use **Queries** to filter and view specific work items.



Adithya B

adithya.b.2024.cse@rajalaksh...

[My Microsoft account](#)

[Switch directory](#)

...



Sign in with a different account

 Azure DevOps

sivas2024cse / DIGITAL CERTIFICATE MANA... / Boards / Work items

DG DIGITAL CERTIFICATE ... +

Overview

Boards

Work items

Boards

Backlogs

Sprints

Queries

Delivery Plans

Analytics views

Work items

Recently updated | + New Work Item | Open in Queries | Column Options | Import Work Items | ...

Filter by keyword

Types | Assigned to | States | Area

ID	Title	Assigned To	Area
20	Verify login with valid credentials	siva s	
14	Certificate Template Design	siva s	
13	Student Data Upload Module	Adithya B	
16	Certificate Verification System	Shabnam Barakkat	
17	Admin Dashboard for Certificate Management	shivam jaiswal	
11	Digital certificate Management with Batch Generation -1	siva s	

Result:

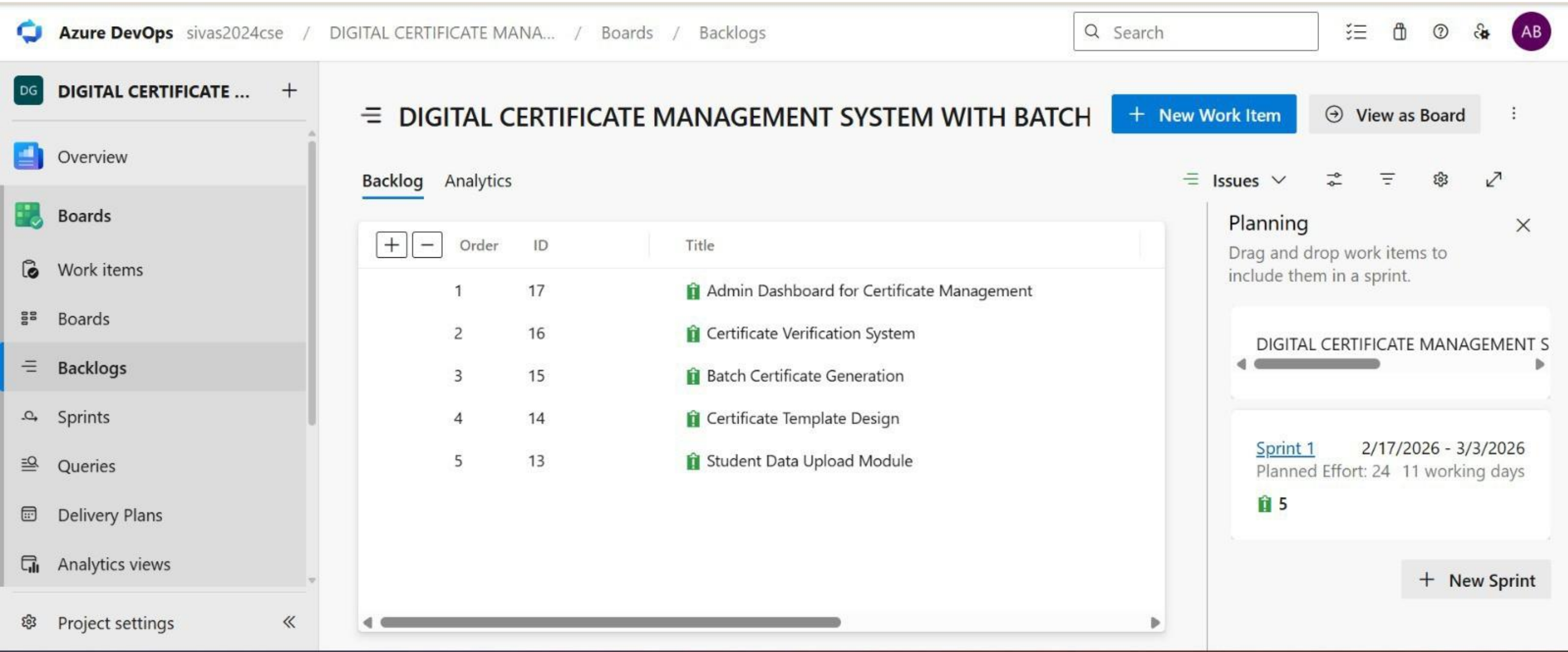
Successfully created an Azure DevOps project with user story management and agile workflow setup .

	SETETXINP NGO U: 3P EPICS, FEATURES, AND USER STORIES PROJECT PLANNING
--	---

Aim:
To learn about how to create epics, user story, features, backlogs for Customer Support Ticket Management System project.

Create Epic, Features, User Stories, Task

- 1.To learn how to create Epics, User Stories, Features, and Backlogs for a Customer Support Ticket Management System project.
- 2.Open Azure DevOps and go to the project dashboard.
- 3.Create or select the **Customer Support Ticket Management System** project.
- 4.Click on **Boards** from the left navigation menu.
- 5.Open the **Backlogs** section.
- 6.Select **Epic** from backlog levels.



- 7.Click **New Work Item** and create an Epic (e.g., Customer Support System).

- 8.Switch to **Feature level** in backlogs.
- 9.Create Features such as Ticket Creation and Ticket Tracking.
- 10. Link Features to the corresponding Epic.
- 11. Switch to **User Stories level**.
- 12. Create User Stories (e.g., “Raise a support ticket”).
- 13. Add acceptance criteria and priority for user stories.
- 14. Link User Stories to Features and organize backlog hierarchy.
- 15. Prioritize work items and arrange them for Agile planning.

Azure DevOps

sivas2024cse

/

DIGITAL CERTIFICATE MANA...

/

Boards

/

Work items

Search

AB

DG

DIGITAL CERTIFICATE ...

+

Overview

Boards

Work items

Boards

Backlogs

Sprints

Queries

Delivery Plans

Analytics views

Project settings

<<

Work Items

← Back to Work Items

NEW EPIC *

Field 'Title' cannot be empty.

Enter title

No one selected

0 Comments

Add Tag

Save

State

To Do

Area

DIGITAL CERTIFICATE MANAGEMENT SYSTEM WITH BAT...

Reason

Added to backlog

Iteration

DIGITAL CERTIFICATE MANAGEMENT SYSTEM WITH BAT...

Details

↔

0

0

Description

Click to add Description.

Discussion

Planning

Priority

2

Start Date

Select a date...

Target Date

Select a date...

Related Work

Add link

Add an existing work item as a parent

Result:
Thus, the creation of epics, features, user story and task has been created successfully.

DG DIGITAL CERTIFICATE ... +

Overview

Boards

Work items

Boards

Boards

Sprints

Delivery Plans

Analytics views

Project settings

DIGITAL CERTIFICATE MANAGEMENT SYSTEM WITH BATCH GENERATION -1

Taskboard Backlog Capacity Analytics

Sprint 1 Person: All

February 17 - March 3
11 work days

Collapse all	To Do	Doing	Done
14 Certificate Template Design To Do siva s			
13 Student Data Upload Module Doing Adithya B			

Aim
To assign user story to specific sprint for Online payment record management system .

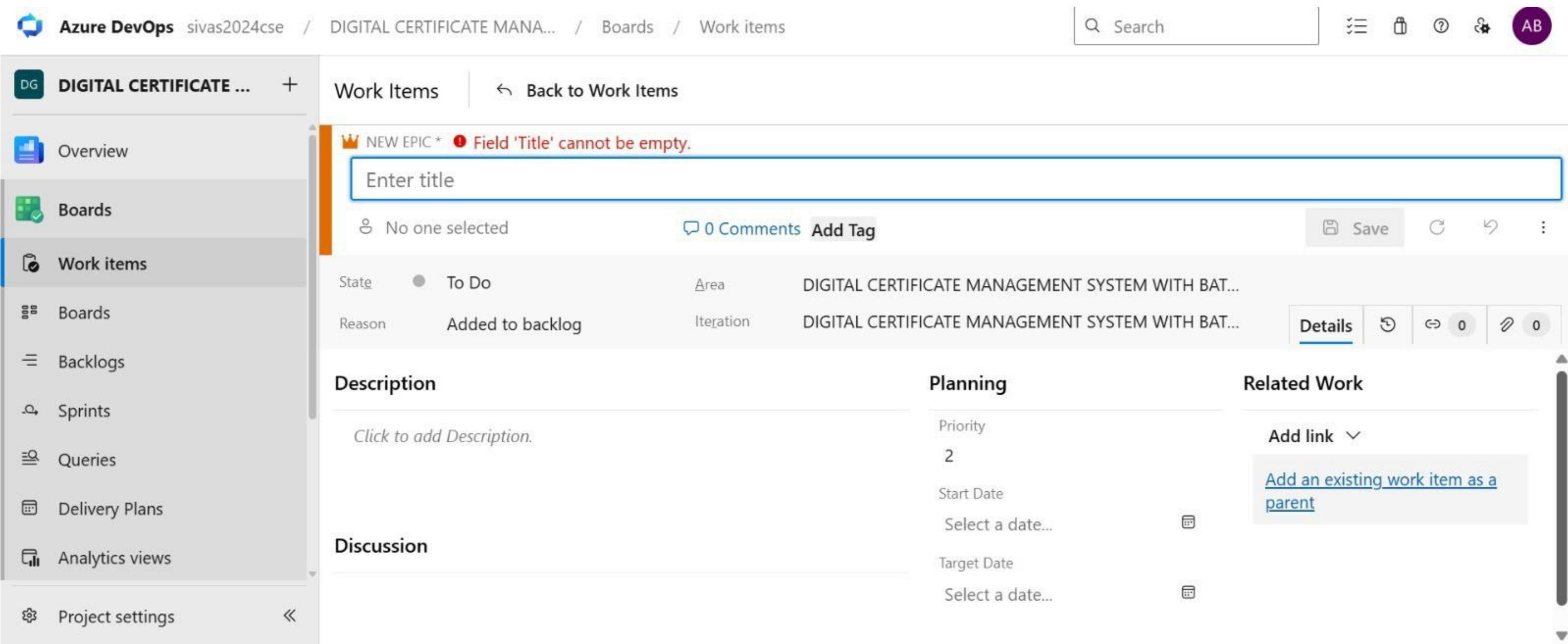
SprintPlan

Result: The Sprints are created for Online payment record management system.

Create Epic, Features, User Stories, Task	EXP NO: 5 POKER ESTIMATION
---	--------------------------------------

Aim:
Create Poker Estimation for the user stories - Online payment record management system .

PokerEstimation



- ☐ Open Azure DevOps and navigate to the project dashboard.
- ☐ Select the project Online Payment Record Management System.
- ☐ Go to Boards → Backlogs from the left navigation menu.
- ☐ Ensure required User Stories are already created for the project.
- ☐ Install or open a Planning Poker / Estimation tool extension if available.
- ☐ Open a User Story (e.g., “Process online payment securely”).

- ☐ Click on Estimate / Story Points option.
- ☐ Invite team members for estimation (if collaboration is enabled).
- ☐ Each member selects a poker card value (e.g., 1, 2, 3, 5, 8).
- ☐ Discuss differences in estimation among team members.
- ☐ Reach a consensus on final story point value.
- ☐ Assign final estimation to the user story.
- ☐ Repeat the process for all user stories in the backlog.

Result:The Estim atio n/Story Points is created for the project using Poker Estim ation.

	EDXPE NSOIG: 6NING CLASS, SEQUENCE AND USE CASES FOR PROJECT ARCHITECTURE
--	---

Aim:

To Design a Class Diagram and Sequence Diagram for Online payment record management System

- 2. Identify the main requirements of the Online Payment Record Management System.
- 3. List the key entities such as User, Payment, Transaction, Account, and Admin.
- 4. Define relationships between entities (association, inheritance, aggregation).
- 5. Open any UML design tool (e.g., draw.io, StarUML, or Visual Paradigm).

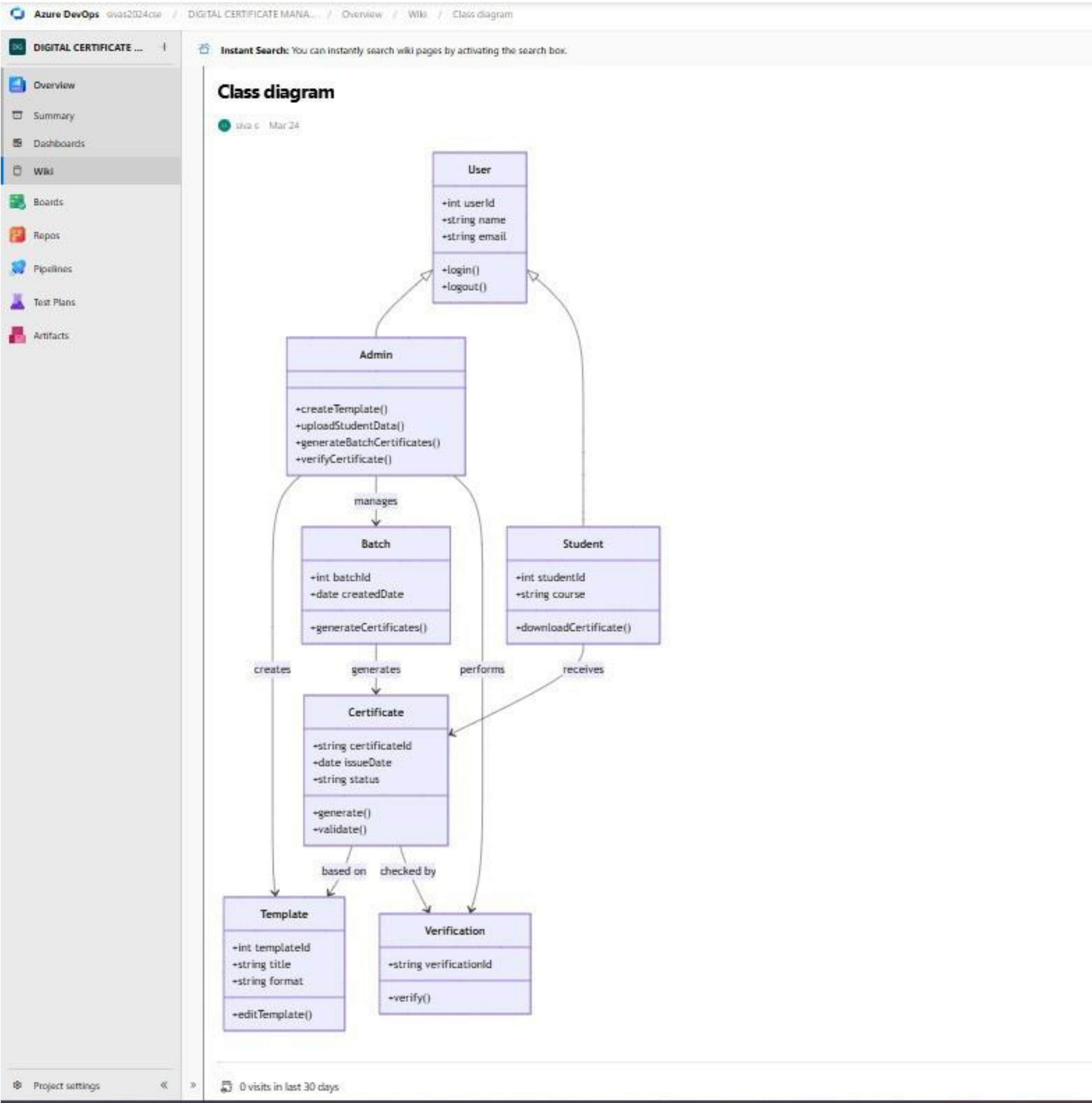
Class Diagram Design

- 6. Create classes for User, Payment, Transaction, and Admin.
- 7. Add attributes to each class (e.g., UserID, Name, PaymentID, Amount).
- 8. Add methods such as makePayment(), verifyTransaction(), and generateReceipt().
- 9. Define relationships between classes using connectors.
- 10. Save and finalize the Class Diagram.

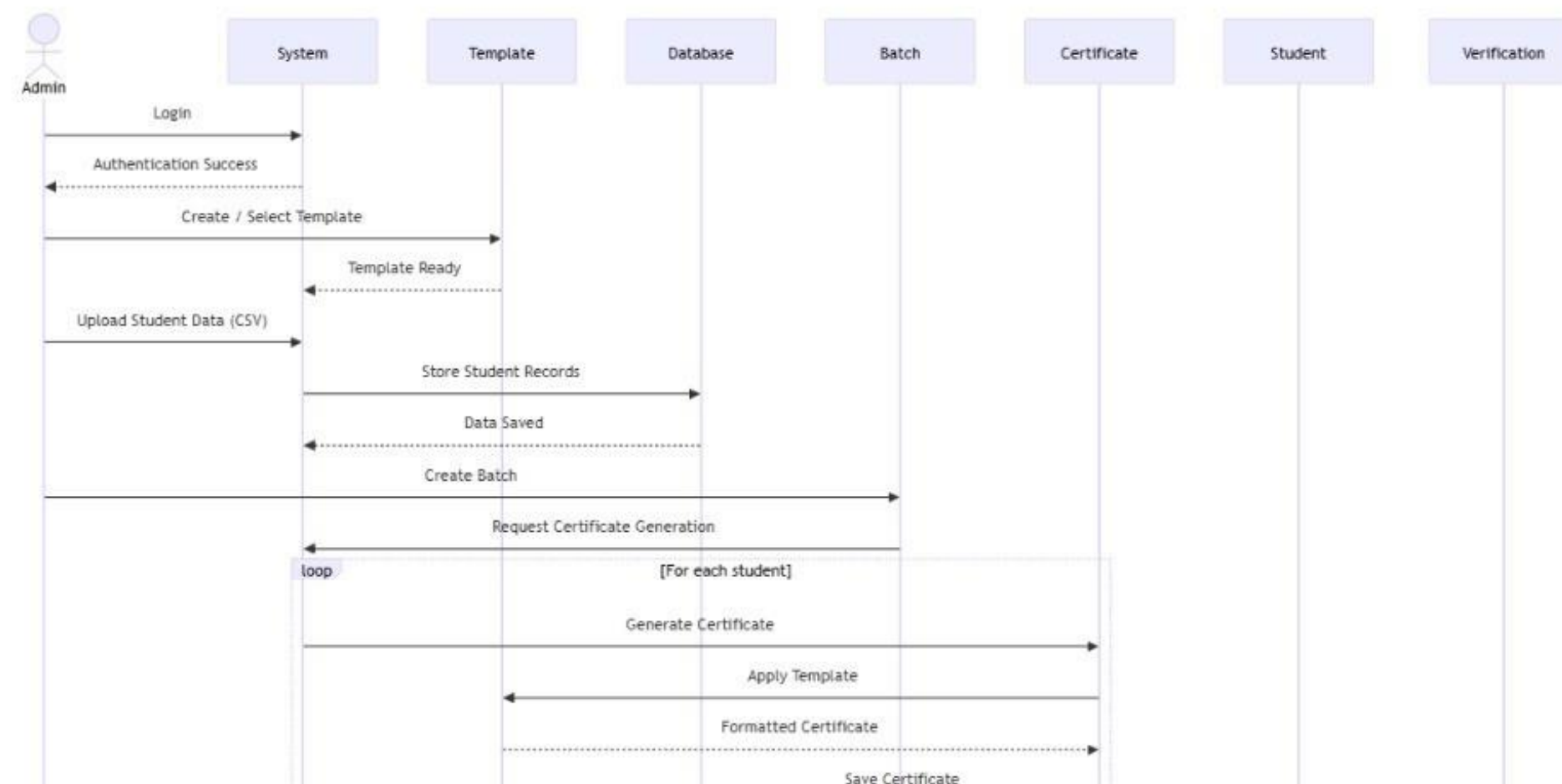
Sequence Diagram Design

- 11. Identify main interaction flow (User initiates payment process).
- 12. Create objects: User, System, Payment Gateway, and Database.
- 13. Draw lifelines for each object.
- 14. Add messages such as login(), requestPayment(), processPayment(), and confirmTransaction().
- 15. Show sequence of interactions step by step.

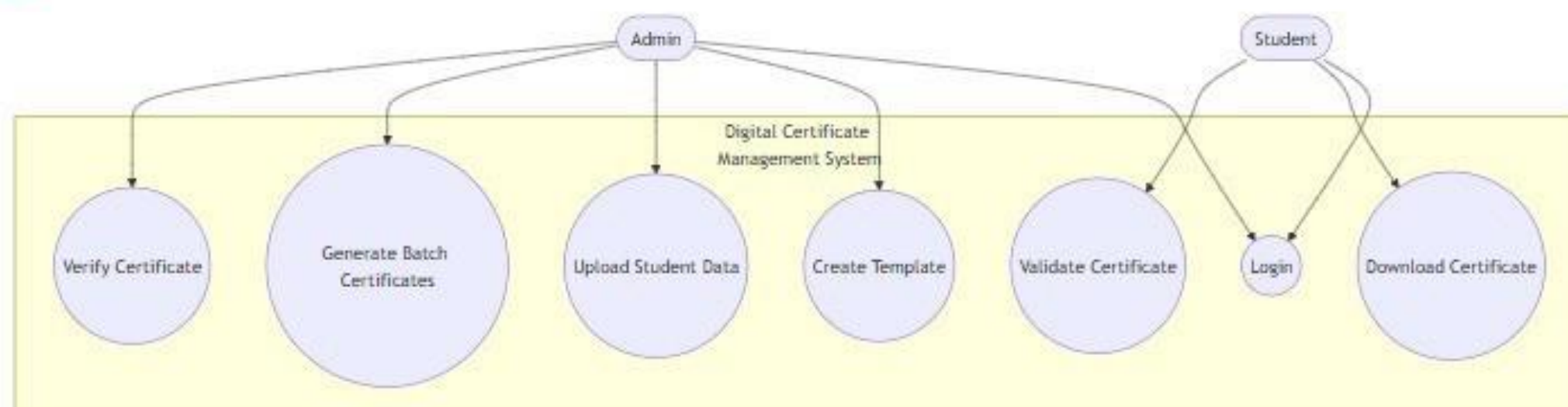
16. Save and finalize the Sequence Diagram.



SS siva s Mar 24



silver Mar 24



0 visits in last 30 days

Comments

AR Add a comment...

Result: The Class Diagram and Sequence Diagram is designed Successfully for Online payment record management system .

	DESEIXGPN NION: G7 ARCHITECTURAL AND ER DIAGRAMS FO PROJECT STRUCTURE
--	--

Aim: To Desig n an Architec tural Diagram and ER Diagram for Online paym ent record management System.

- 2. Identify the requirements of the Online Payment Record Management System.
- 3. List the main components such as User Interface, Application Server, Database, and Payment Gateway.
- 4. Identify entities required for the system such as User, Account, Transaction, Payment, and Admin.

Architectural Diagram Design

- 5. Open a diagram tool such as draw.io, StarUML, or Visual Paradigm.
- 6. Draw the **Client layer (User Interface)** for user interaction
- 7. Add the **Application layer (Business Logic)** for processing requests.
- 8. Add the **Database layer** for storing records.
- 9. Include external service like **Payment Gateway**.
- 10. Connect all layers to show system flow and interaction.
- 11. Finalize and save the Architectural Diagram.

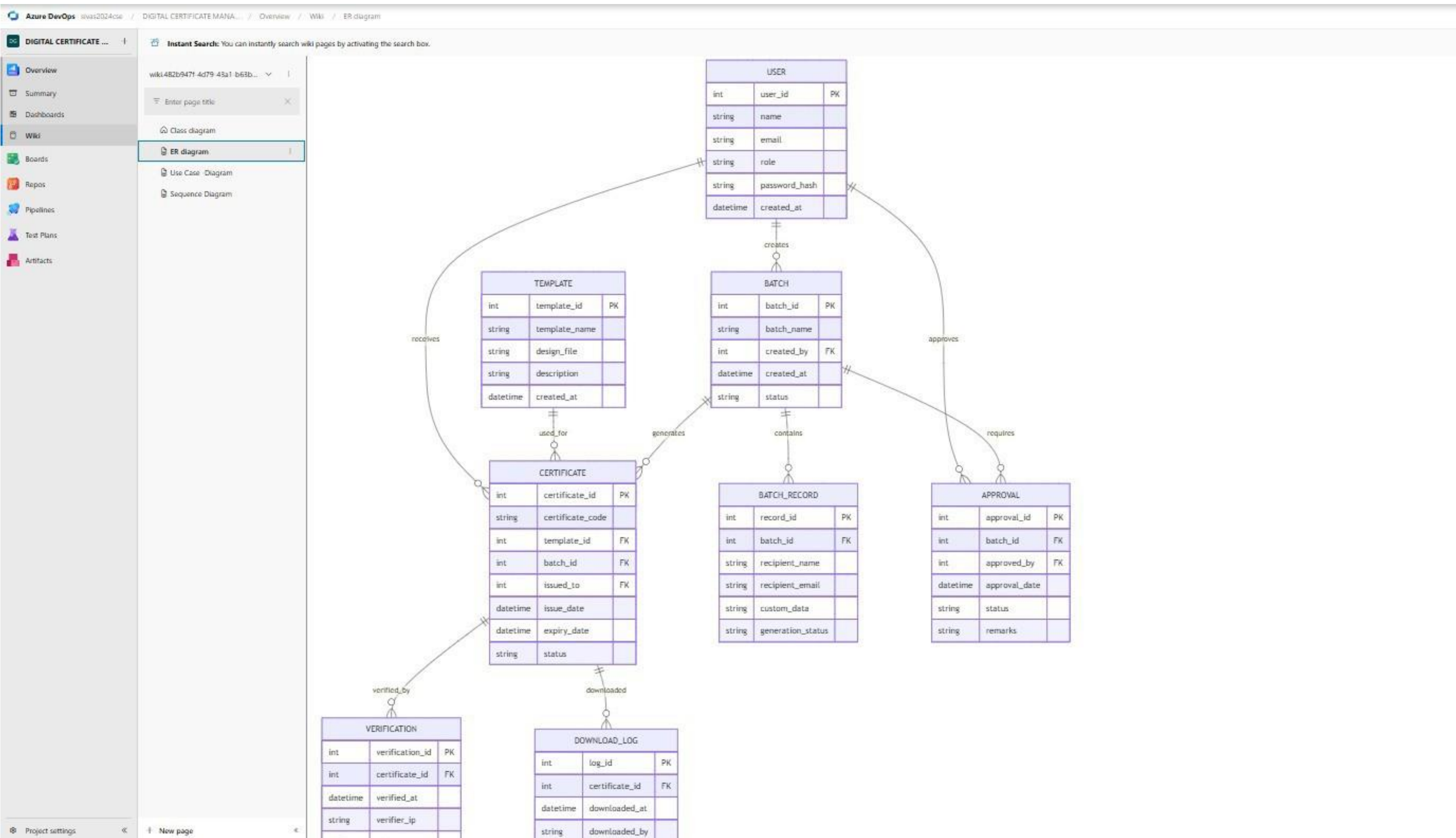
ER Diagram Design

- 12. Identify entities such as User, Account, Transaction, Payment, and Admin.
- 13. Define attributes for each entity (e.g., UserID, Name, PaymentID, Amount).
- 14. Identify relationships between entities (one-to-one, one-to-many).
- 15. Create ER symbols: rectangle for entity, oval for attributes, diamond for relationships.
- 16. Draw relationships such as User makes Payment, Payment generates Transaction.

17. Add primary keys and foreign keys for database structure.

18. Finalize and save the ER Diagram.

7B.ERDiagram



Result:

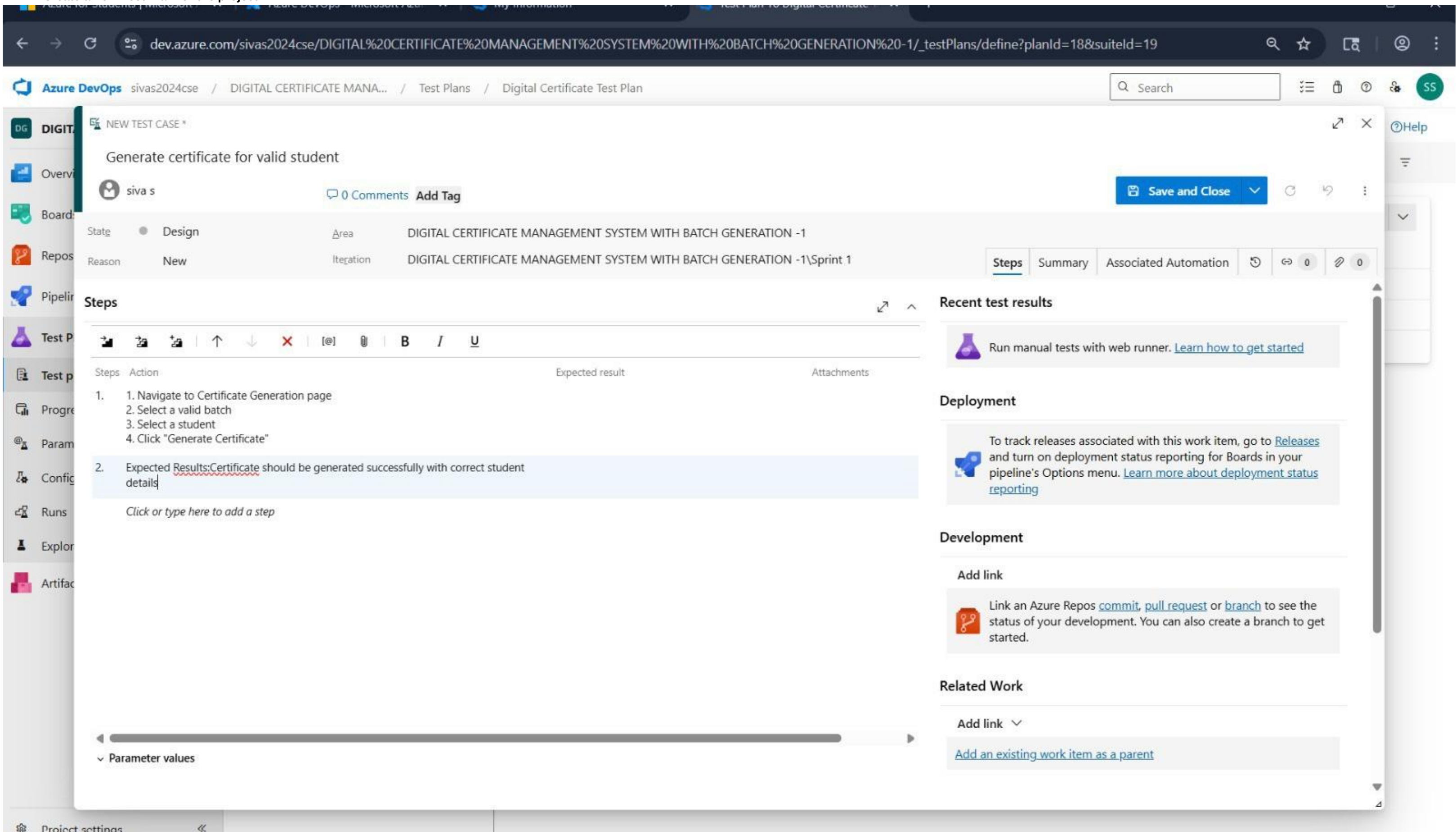
The Architecture Diagram and ER Diagram is designed Successfully for Online payment record management system .

EXP TESTING – TEST PLANS AND TEST CASES

Aim: Test Plans and Test Case and write two test cases for at least five user stories showcasing the happy path and error scenarios in azure DevOps platform .

Test Planning and Test Case

2. pen Azure DevOps and select the project.
3. Navigate to Test Plans from the left menu.
4. Create a new Test Plan for the project.



5. Name the test plan (e.g., Online Payment System Testing).
6. Create Test Suites for each user story.
7. Add user stories related to the Online Payment Record Management System.
8. Define testing scope, objectives, and environment.

User Stories and Test Cases

User Story 1: User Login

- 9. Test Case 1 (Happy Path): Valid username and password → Login successful.
- 10. Test Case 2 (Error Scenario): Invalid password → Error message displayed.

User Story 2: Make Payment

- 11. Test Case 1 (Happy Path): Valid payment details → Payment successful.
- 12. Test Case 2 (Error Scenario): Insufficient balance → Payment failed.

User Story 3: View Transaction History

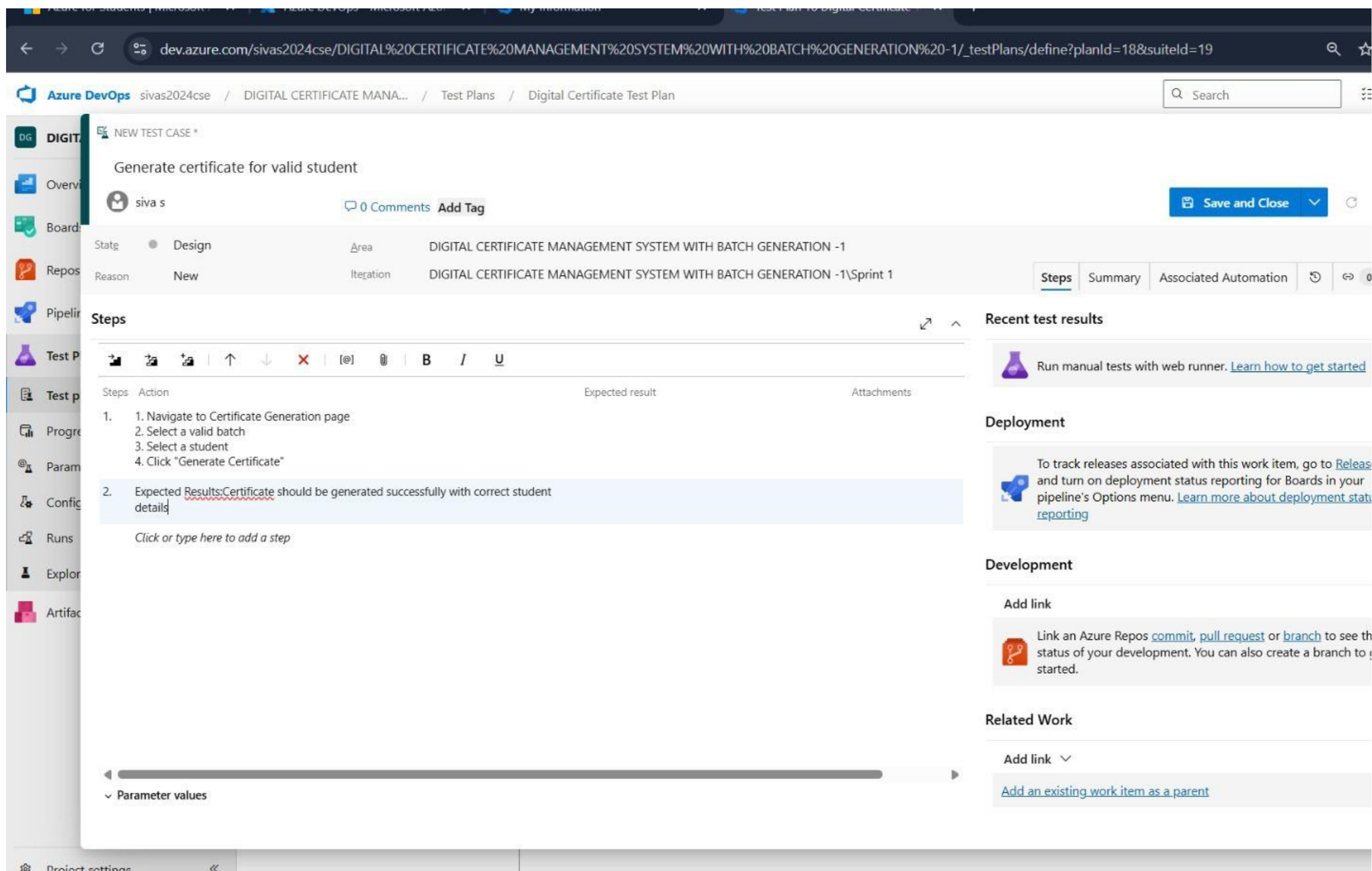
- 13. Test Case 1 (Happy Path): User requests history → Transactions displayed.
- 14. Test Case 2 (Error Scenario): No transactions available → “No records found” message.

User Story 4: Add Bank Account

- 15. Test Case 1 (Happy Path): Valid bank details → Account added successfully.
- 16. Test Case 2 (Error Scenario): Invalid account number → Error message shown.

User Story 5: Generate Receipt

- 17. Test Case 1 (Happy Path): Completed payment → Receipt generated successfully.
- 18. Test Case 2 (Error Scenario): Payment not completed → Receipt not generated.



Result:The test plans and test cases for the user stories is created in Azure Dev Ops with Happy Path and Error Path

	EXP NO: 9LOAD TESTING AND PIPELINES
--	-------------------------------------

Aim:

To create an Azure Load Testing resource and run a load test to evaluate the performance of a target endpoint and to create and demonstrate an Azure DevOps pipeline for automating application builds, tests, and deployment.

Load Testing

Azure Load Testing:

Azure Load Testing allows you to simulate high traffic and stress tests for your web applications and APIs to understand how they perform under load. It helps identify performance bottlenecks, scalability issues, and optimize resource usage before deployment.

Steps to Create an Azure Load Testing Resource:

Before you run your first test, you need to create the Azure Load Testing resource:

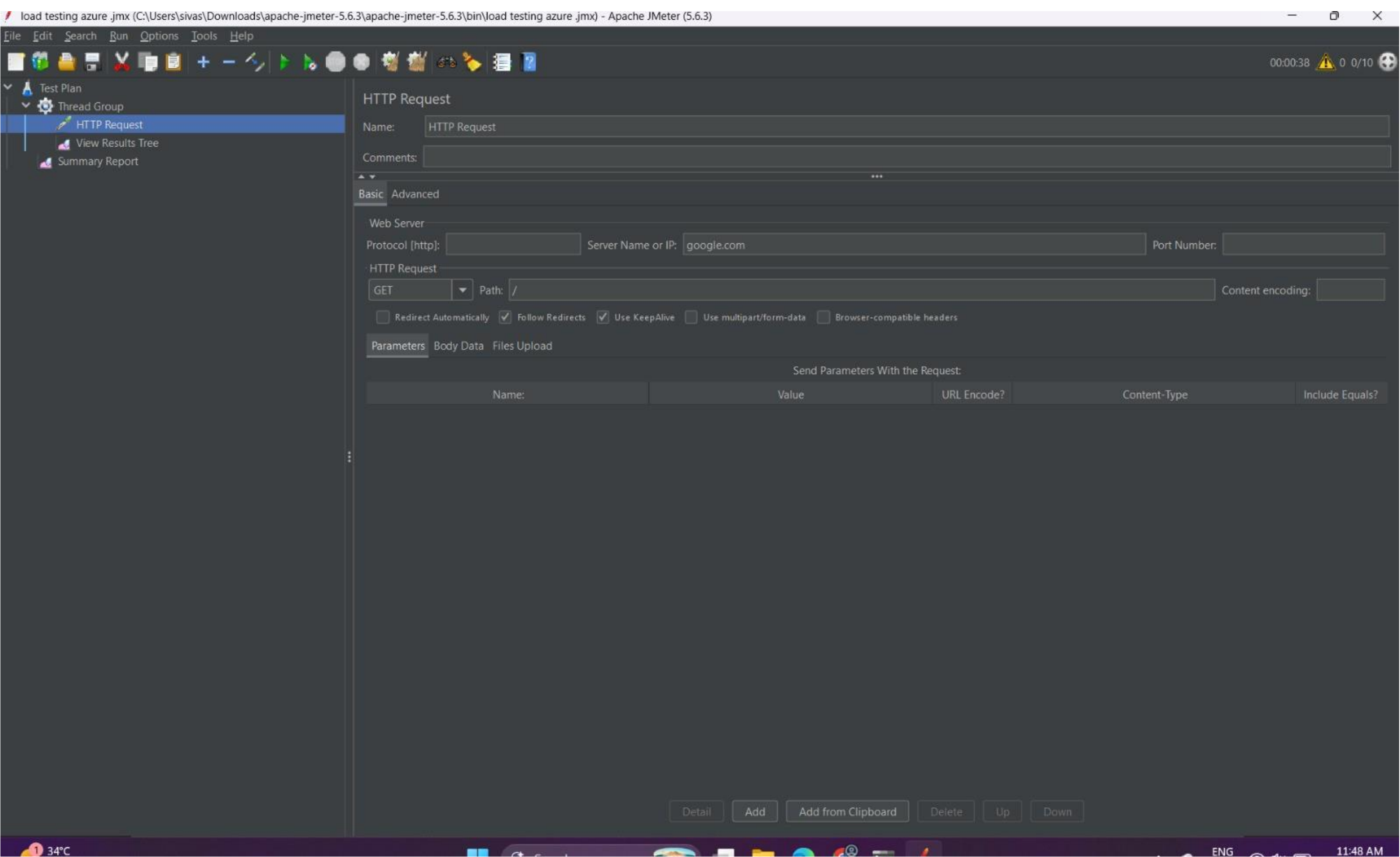
1. Sign in to Azure Portal
Go to <https://portal.azure.com> and log in.
2. Create the Resource
 - o *Goto Create a resource* → Search for “Azure Load Testing”.
 - o Select Azure Load Testing and click Create.
3. Fill in the Configuration Details
 - o *Subscription*: Choose your Azure subscription.
 - o *Resource Group*: Create new or select an existing one.
 - o *Name*: Provide a unique name (no special characters).
 - o *Location*: Choose the region for hosting the resource.
4. (Optional) Configure tags for categorization and billing.
5. Click Review + Create, then Create.
6. Once deployment is complete, click Go to resource.

Steps to Create and Run a Load Test:

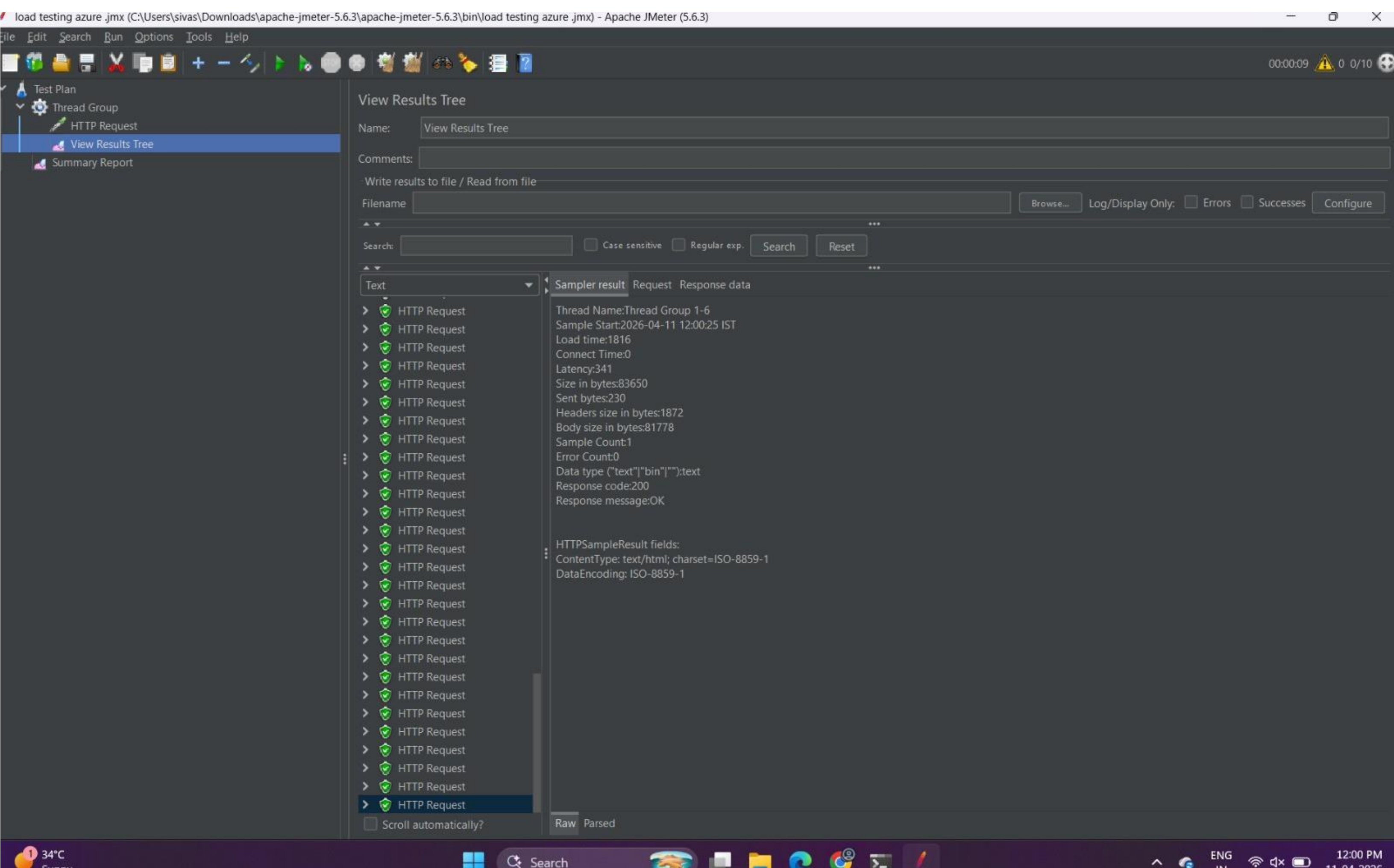
Once your resource is ready:

1. Go to your Azure Load Testing resource and click Add HTTP requests > Create.
2. **Basics Tab**
 - o *Test Name*: Provide a unique name.
 - o *Description*: (Optional) Add test purpose.
 - o *Run After Creation*: Keep checked.
3. **Load Settings**
 - o *Test URL*: Enter the target endpoint (e.g., <https://yourapi.com/products>).

4 Click Review+Create → Create to start the test.



Load Testin



Description:

This experiment demonstrates how to connect a GitHub-hosted Flask-based music recommendation project with Azure DevOps. The pipeline will automatically install dependencies, run basic tests, and publish artifacts. This ensures that every commit triggers checks for reliability and smooth deployment.

Steps:

1. Connect GitHub to Azure DevOps:

- o In Azure DevOps, create a new project.
 - o Create a pipeline and select GitHub as the source.
 - o Authorize access to your GitHub repository, ensuring that Azure DevOps can pull the repository for your pipeline.
2. Create azure-pipelines.yml in Your Repo Root:
- o In your GitHub repository, create a new file called azure-pipelines.yml in the root directory.
 - o Add the following basic pipeline configuration for Python and Flask:

yamlCode

name: Payment System CI

on:

push:

branches: ["main"]

jobs:

build:

runs-on:

ubuntu-latest steps:

- uses: actions/checkout@v3

- name: Run Build

Script run: |

echo "Building the Online Payment System..."

echo "Validating HTML and JS files..."

3. Pipeline Tasks Include:

- o Setting up the Python environment using the UsePythonVersion task.
- o Installing project dependencies from project/requirements.txt. Make sure the path to requirements.txt is correct (it is located under the project folder).
- o Running a simple Python script to verify that Python is set up correctly and the pipeline works.

4.Run and Monitor Pipeline:

- o Commit changes to the main branch of your repository to trigger the pipeline in Azure DevOps.
- o Monitor the logs in the Azure DevOps portal to view logs, errors, or success messages and ensure everything runs smoothly.

Pipeline

← Payment System CI

✔ Create main.yml #1

Re-run all jobs

...

Summary

All jobs

build

Run details

Usage

Workflow file

Annotations

1 warning

build

succeeded 17 hours ago in 4s

Search logs

Refresh

Settings

> ✔ Set up job

0s

> ✔ Run actions/checkout@v3

1s

> ✔ Run Build Script

0s

> ✔ Post Run actions/checkout@v3

0s

> ✔ Complete job

0s

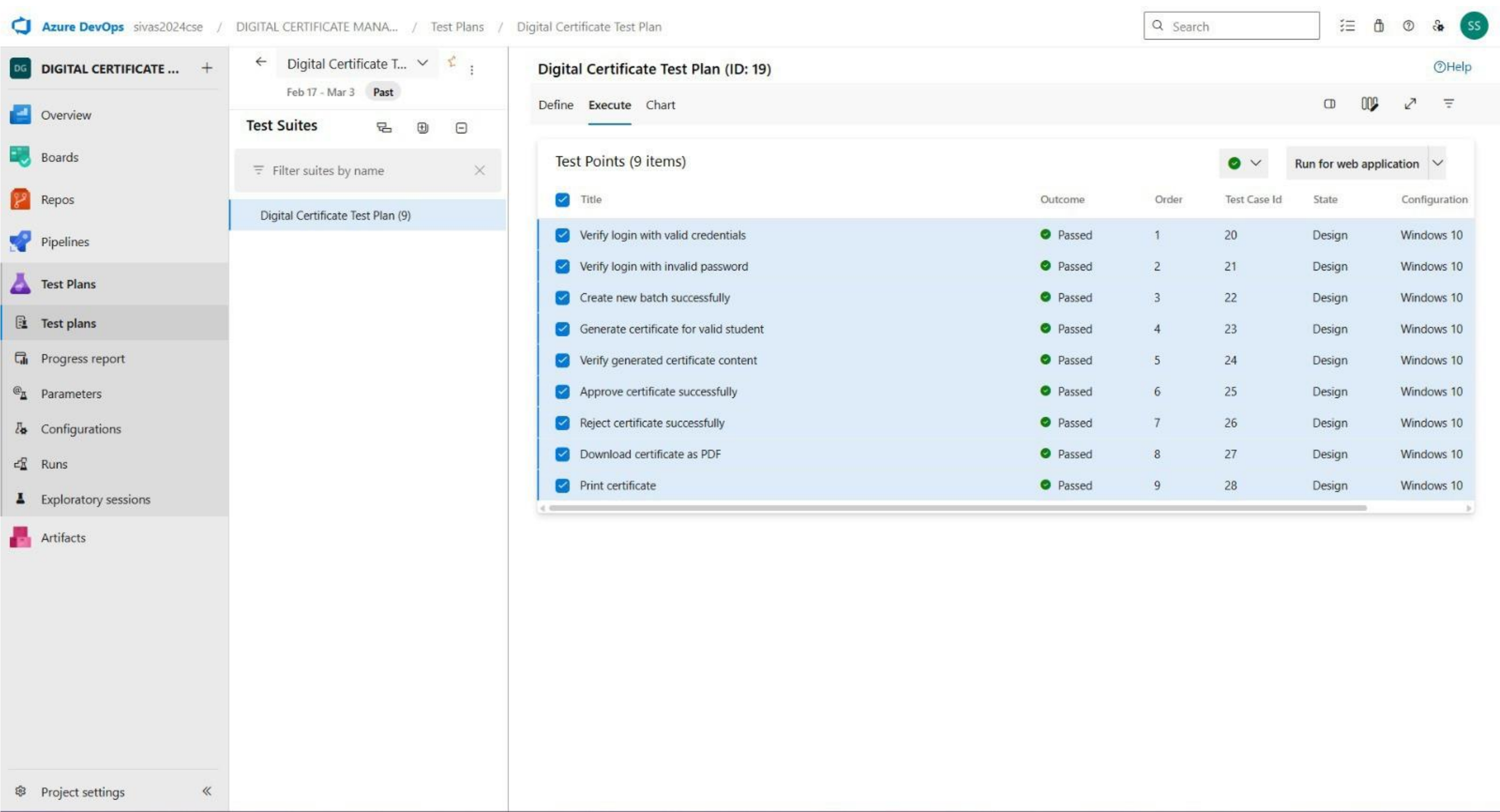
Result:

Successfully created the Azure Load Testing resource and executed a load test to assess the performance of the specified endpoint and also demonstrated pipelines in azure devops

Create Epic Features User	EXPT010B: PROJECT STRUCTURE & NAMING CONVENTION
---------------------------	---

Aim:
To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand , navigate, and extend Online payment record management system .

GitHub ProjectStructure



2. Create a new repository in GitHub for the project.
3. Name the repository (e.g., Online-Payment-Record-Management-System).
4. Initialize the repository with a README file.
5. Define the main project folders such as:
 - src (source code)
 - docs (documentation)
 - tests (test cases)
 - config (configuration files)

- assets (images, resources)

Folder Structure Design

6. Organize backend code inside the src/backend folder.
7. Organize frontend code inside the src/frontend folder.
8. Store test cases inside the tests folder.
9. Store documentation like diagrams and reports inside the docs folder.
10. Store configuration files (database, API settings) inside the config folder.

File Naming Conventions

11. Use meaningful and consistent file names.
12. Use lowercase letters and hyphens (e.g., user-login.js, payment-service.js).
13. Separate modules based on functionality.
14. Keep class names in PascalCase (e.g., PaymentController).
15. Keep constants in uppercase (e.g., API_KEY).

Project Structure Example

16. Maintain a structured hierarchy for easy navigation and scalability.
17. Ensure separation of concerns between modules.
18. Add a README file explaining the project structure and setup instructions.

Result:

The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the codebase.